

DOCUMENTACION FUNCIONAL

Proyecto: VTT Tool/VTT2.py

Fecha: 06/05/2026

Descripción técnica del funcionamiento de la UI, filtros, timeline, VTT Summary y exportaciones.

TABLA DE CONTENIDO

TABLA DE CONTENIDO.....	2
1. OBJETIVO GENERAL	4
2. FUENTE DE DATOS.....	4
3. FLUJO GENERAL DE LA APP.....	5
3.1. Carga inicial.....	5
3.2. Construccion de la seleccion	5
3.3. Seleccion de la fila activa	6
3.4. Render de la UI	6
4. COMO FUNCIONAN LOS FILTROS	6
4.1. Filtro POL	6
4.2. Filtro POD.....	6
4.3. Filtro de registro cuando hay duplicados POL/POD	6
5. CAMPOS VISIBLES EN LA UI Y SU ORIGEN	7
5.1. Tarjetas superiores	7
5.2. Campos mostrados debajo del resumen	7
5.3. Comportamiento de render_box.....	7
6. COMO SE CONSTRUYE LA TIMELINE.....	7
6.1. Eje de fechas.....	7
6.2. Encabezados	7
6.3. Logica general de pintado	8
7. CALCULO DE CADA FILA DE LA TIMELINE	8
8. NOTA SOBRE "16 MANUFACTURING"	11
9. COMO SE CALCULA EL BLOQUE "VTT SUMMARY"	11
9.1. CUSTOMER LEADTIME (CLT).....	11
9.2. Transportation Duration.....	11
9.3. SUPPLIER>POL	11
9.4. POL>POD.....	11
9.5. POD DETENTION	12
9.6. POD>PLANT	12
10. CUSTOMER SAFETY STOCK	12
11. CAPTURA OCULTA PARA EXPORTACION	12
12. COMO FUNCIONA EL BOTON "GENERATE FILES"	12

13. EXPORTACION A IMAGEN.....	13
14. EXPORTACION A EXCEL.....	13
14.1. Hoja Timeline.....	13
14.2. Hoja visual "UI Snapshot"	13
15. TOLERANCIA A DATOS FALTANTES Y VARIANTES DE NOMBRE.....	14
16. RESUMEN CORTO DEL COMPORTAMIENTO.....	14
17. PUNTOS IMPORTANTES A TENER EN CUENTA SI SE MODIFICA EL EXCEL O EL CODIGO.....	14

1. OBJETIVO GENERAL

VTT2.py es una app de Streamlit que lee el archivo Excel "VTT DATA.xlsx" y construye una vista tipo timeline/Gantt para una combinacion de POL y POD. La pantalla principal muestra:

- Filtros de seleccion.
- Datos generales del registro seleccionado.
- Una tabla timeline con columnas Steps, Day, Day+ y Final Day.
- Un resumen KPI llamado "VTT SUMMARY".
- El campo "Customer Safety STOCK".
- Los campos "Commodity" y "E/D".
- Un boton "Generate files" para descargar Excel e imagen.

El archivo se ejecuta de arriba hacia abajo. Cada rerun de Streamlit recalcula la fila seleccionada, la tabla, el resumen KPI y los archivos de exportacion.

2. FUENTE DE DATOS

La unica fuente principal es:

- VTT Tool/VTT DATA.xlsx

El codigo carga ese archivo con pandas usando:

- `pd.read_excel(vtt_data_path)`

La app no consulta bases de datos ni APIs externas. Todo lo que se ve en la UI sale del Excel seleccionado al iniciar la app.

Columnas del Excel que VTT2.py usa directamente o indirectamente:

- ID
- POL
- POD
- Carrier
- Name Destin Site
- Expiration Date
- Safety stock
- 1 Day Customer Order
- 2 Day ILN Order
- 3 First Receipt Days
- 3 .1 Time of Recept in ILN
- 3.2 First Receipt Days
- 4.1 Packaging preparation/loading
- 4.2 Packaging preparation/loading

- 4.3 Packaging preparation/loading
- 5.1 Transport ILN to POL
- 5.2 Transport ILN to POL
- 5.3 Transport ILN to POL
- First Day to POL
- 6 First Day to POL
- 7 Cutt off
- 8 ETD
- Transit time
- 9 ETD>ETA o 9 ETD> ETA
- Time for security
- 10 Days flexibility 1
- Time for security2 buffer
- 11 Days flexibility 2
- Cust.
- 12 Customs Clearance o 12 Customs clearence
- Trpt POD/PFI vers Usine
- 13 Transport to Plant
- Round. o Round
- 14 Rounding
- 15 Due Date
- 16 Manufacturing
- Comodity o Commodity

Dos campos visibles no se buscan por nombre sino por posicion de columna:

- ILN/FF se toma de `df_vtt.columns[8]`. En el Excel actual esa columna es " ILN/FF".
- Shipper se toma de `df_vtt.columns[10]`. En el Excel actual esa columna es " Shipper".

Esto significa que si el orden de columnas cambia, esos dos campos pueden apuntar a una columna incorrecta aunque el nombre exista.

3. FLUJO GENERAL DE LA APP

3.1. Carga inicial

- Se importa pandas, streamlit, openpyxl, PIL y utilidades de fecha.
- Se carga VTT DATA.xlsx.
- Se configura la pagina en layout wide.
- Se inyecta CSS para reducir margenes y escribir los encabezados de fecha en vertical.

3.2. Construccion de la seleccion

- Se crean dos columnas principales: una angosta para filtros y una ancha para la timeline.

- En la columna izquierda se crean los filtros POL y POD.
- La seleccion actual vive en st.session_state con claves pol_select, pod_select y, si hace falta, record_select.

3.3. Seleccion de la fila activa

- Primero se filtra el dataframe por POL y POD.
- Si el filtro devuelve una sola fila, esa fila se usa como row.
- Si devuelve varias filas, aparece un tercer selector "Registro (varios coincidieron)".
- La etiqueta de ese selector concatena datos utiles para distinguir registros:
 - ID-Cartography + Carrier + Plant + Expiration Date.
- Si no hay coincidencias, row queda en None.

3.4. Render de la UI

Con row ya definida, el script arma:

- Tarjetas informativas superiores.
- La tabla timeline HTML.
- El bloque KPI "VTT SUMMARY".
- Customer Safety STOCK.
- Slider de dias visibles.
- Bloque oculto para captura de imagen/Excel.
- Botones de descarga.

4. COMO FUNCIONAN LOS FILTROS

4.1. Filtro POL

- Se alimenta con los valores únicos no nulos de la columna POL.
- Si session_state no tiene pol_select, toma el primer POL disponible.

4.2. Filtro POD

- No muestra todos los POD del archivo.
- Solo muestra los POD que pertenecen al POL actualmente seleccionado.
- Si al cambiar POL el POD viejo ya no existe, el codigo fuerza pod_select al primer POD valido.

4.3. Filtro de registro cuando hay duplicados POL/POD

- Si el par POL/POD tiene mas de una fila, aparece un selectbox adicional.
- La opcion seleccionada guarda el indice real del dataframe.
- La fila activa queda en filtered_df.loc[selected_label].

5. CAMPOS VISIBLES EN LA UI Y SU ORIGEN

5.1. Tarjetas superiores

- ID-Cartography: columna ID.
- Carrier: columna Carrier.
- Shipper: columna en posicion 10 del Excel.
- ILN/FF: columna en posicion 8 del Excel.
- PLANT: columna Name Destin Site.

5.2. Campos mostrados debajo del resumen

- Customer Safety STOCK: columna Safety stock.
- Commodity: columna Commodity si existe; si no, Comodity.
- E/D: columna Expiration Date formateada a dd/mm/YYYY cuando se puede convertir a fecha.

5.3. Comportamiento de render_box

Los campos mostrados con render_box:

- Se dibujan como HTML manual, no como widgets nativos de Streamlit.
- Truncan visualmente textos largos.
- Guardan el valor completo en el atributo title para verlo como tooltip.

6. COMO SE CONSTRUYE LA TIMELINE

6.1. Eje de fechas

- today = fecha actual del sistema.
- start_date = lunes de la semana actual.
- timeline_days = lista de fechas consecutivas desde ese lunes.
- La cantidad de dias viene de days_slider_timeline.
- El slider permite entre 7 y 150 dias.

6.2. Encabezados

La tabla tiene cuatro columnas fijas:

- Steps
- Day
- Day+
- Final Day

Luego agrega una columna por cada dia de timeline_days.

Detalles visuales:

- Las semanas se agrupan por numero ISO y se muestran como WeekX ejem.W21
- Sabados y domingos se pintan en rojo claro.
- Dias habiles se pintan en azul muy claro en el encabezado.
- En el encabezado diario solo se muestra la inicial del dia.

6.3. Logica general de pintado

Para casi todas las filas, el rango coloreado se calcula asi:

- `final_day` = valor de la columna Final Day.
- `day_plus` = valor de Day+.
- `paint_len` = `day_plus` si `day_plus > 0`; si no, 1.
- `start_idx` = `max(1, final_day - paint_len + 1)`.

Luego se colorea desde `start_idx` hasta `final_day`.

Excepciones:

- Steps 1, 2, 6, 7 y 8 siempre pintan un solo dia, aunque Day+ sea 0.
- Step 9 usa azul en vez de verde y dibuja un icono de barco.

7. CALCULO DE CADA FILA DE LA TIMELINE

IMPORTANTE:

- "Day" representa el dia de inicio del paso.
- "Day+" representa la duracion o buffer del paso.
- "Final Day" representa el ultimo dia del paso.
- Si una columna no existe o viene vacia, la UI intenta mostrar "-" o 0 segun el caso.
- La funcion `_coerce_to_int` convierte numeros, strings con texto y celdas mixtas a entero. Si falla, devuelve 0.

Step 1. Day Customer Order

- Day: columna 1 Day Customer Order.
- Day+: fijo en 0.
- Final Day: columna 1 Day Customer Order.
- Pintado: solo el dia final.

Step 2. Day ILN/FF Order

- Day: columna 2 Day ILN Order.
- Day+: fijo en 0.
- Final Day: columna 2 Day ILN Order.
- Pintado: solo el dia final.

Step 3. First Receipt Days

- Day: columna 3 First Receipt Days.
- Day+: columna 3 .1 Time of Receipt in ILN.
- Final Day: columna 3.2 First Receipt Days.
- Pintado: desde Final Day - Day+ + 1 hasta Final Day.

Step 4. Pack. prep. & load

- Day: columna 4.1 Packaging preparation/loading.
- Day+: columna 4.2 Packaging preparation/loading.
- Final Day: columna 4.3 Packaging preparation/loading.
- Pintado: desde Final Day - Day+ + 1 hasta Final Day.

Step 5. Transport to POL

- Day: columna 5.1 Transport ILN to POL.
- Day+: columna 5.2 Transport ILN to POL.
- Final Day: columna 5.3 Transport ILN to POL.
- Pintado: desde Final Day - Day+ + 1 hasta Final Day.

Step 6. First Day to POL

- Day: columna 6 First Day to POL.
- Day+: columna First Day to POL.
- Final Day: columna 6 First Day to POL.
- Pintado: en la práctica un solo día porque este step está en el grupo de excepciones que fuerza paint_len = 1.

Step 7. Cut off

- Day: columna 7 Cutt off.
- Day+: fijo en 0.
- Final Day: columna 7 Cutt off.
- Pintado: solo el dia final.

Step 8. ETD

- Day: columna 8 ETD.
- Day+: fijo en 0.
- Final Day: columna 8 ETD.
- Pintado: solo el dia final.

Step 9. Transit Duration (ETD>ETA)

- Day: columna 8 ETD.
- Day+: columna Transit time.
- Final Day: columna 9 ETD> ETA si existe; si no, 9 ETD>ETA.
- Pintado: rango azul desde Final Day - Transit time + 1 hasta Final Day.
- Marca visual especial: icono de barco.

Step 10. Days flexibility 1

- Day: Final Day del step 9 + 1.
- Day+: columna Time for security.
- Final Day: columna 10 Days flexibility 1 si existe y trae dato.
- Si 10 Days flexibility 1 no existe o esta vacia, el codigo la deriva asi:
 - Final Day step 9 + 1 + Time for security.

Step 11. Days flexibility 2

- Day: Final Day del step 10 + 1.
- Day+: columna Time for security2 buffer.
- Final Day: columna 11 Days flexibility 2.

Step 12. Customs clearence

- Day: Final Day del step 11 + 1.
- Day+: columna Cust.
- Final Day: columna 12 Customs Clearance; si no existe, usa 12 Customs clearence.

Step 13. Transport to plant

- Day: Final Day del step 12 + 1.
- Day+: columna Trpt POD/PFI vers Usine.
- Final Day: columna 13 Transport to Plant.

Step 14. Rounding

- Day: Final Day del step 13 + 1.
- Day+: columna Round.; si no existe, usa Round.
- Final Day: columna 14 Rounding.

Step 15. Due Date

- Day: Final Day del step 14 + 1.
- Day+: valor calculado por `_due_date_day_plus_value(row, df_vtt)`.
- Final Day: columna 15 Due Date.

Regla de `_due_date_day_plus_value`:

- Por defecto devuelve 7.
- Excepcion especial: si ID = 0001, POL = CNSHA y POD = PTLEI, devuelve 5.

8. NOTA SOBRE "16 MANUFACTURING"

El codigo auxiliar de exportacion si tiene soporte para:

- Day+: 7.
- Final Day: columna 16 Manufacturing.

Pero la lista visual `time_labels` termina en el Step 15. Eso significa que, en el estado actual del script, "16 Manufacturing" no aparece en la timeline visible ni en el loop principal del Excel Timeline, aunque existan helpers preparados para ese paso.

9. COMO SE CALCULA EL BLOQUE "VTT SUMMARY"

El resumen KPI se arma con la funcion `_build_kpi_rows`. Cada KPI devuelve:

- etiqueta
- valor numerico
- `start_day`, usado para dibujar su barra en el mini Gantt

VTT SUMMARY (KPI):

9.1. CUSTOMER LEADTIME (CLT)

- Formula: Final Day de 14 Rounding - Final Day de 1 Day Customer Order + 1.
- Barra: comienza donde empieza el step 1.

9.2. Transportation Duration

- Formula: Final Day de 14 Rounding - valor Day de 5.1 Transport ILN to POL + 1.
- Barra: comienza donde empieza el step 5.

9.3. SUPPLIER>POL

- Formula: Final Day de 8 ETD - valor Day de 3 First Receipt Days + 1.
- Barra: empieza en 3 First Receipt Days.

9.4. POL>POD

- Formula: Transit time + Time for security.
- Ambos valores se convierten con `pd.to_numeric` y luego a entero.
- Barra: empieza justo despues del tramo SUPPLIER>POL, en modo secuencial.
- Color: azul claro.

9.5. POD DETENTION

- Formula: Final Day de 12 Customs Clearance - Final Day de 10 Days flexibility 1.
- Barra: empieza despues de POL>POD.

9.6. POD>PLANT

- Formula: Final Day de 14 Rounding - Final Day de 12 Customs Clearance.
- Barra: empieza despues de POD DETENTION.

10. CUSTOMER SAFETY STOCK

- Se extrae de la columna Safety stock.
- Se muestra debajo del bloque VTT SUMMARY.
- Tambien se agrega al Excel.
- Tambien se incluye en la captura de imagen.

11. CAPTURA OCULTA PARA EXPORTACION

Antes de los botones de descarga, el script construye un HTML oculto llamado timeline_capture. Ese bloque incluye:

- Titulo "VTT View".
- POL, POD y Days to Show.
- Datos ID, Carrier, Shipper, ILN/FF y PLANT.
- La timeline completa.
- El bloque VTT SUMMARY.
- Customer Safety STOCK.
- E/D.
- Commodity.

Ese bloque no se muestra al usuario, pero sirve como fuente comun para la exportacion PNG y para la hoja visual del Excel.

12. COMO FUNCIONA EL BOTON "GENERATE FILES"

Cuando se pulsa el boton:

- El servidor genera un workbook base con build_excel_workbook(...).
- El nombre del archivo se arma asi:
 - VTT_{POL}_{POD}_{SHIPPER}
- Si alguno de esos valores esta vacio, el codigo usa UNKNOWN.

Despues aparecen dos botones para generar la descarga de los archivos:

- Excel file
- Image

13. EXPORTACION A IMAGEN

El boton "Image" usa html2canvas en el navegador para capturar timeline_capture y descargarlo como PNG.

Caracteristicas:

- Fondo blanco.
- Escala 2 para mayor resolucion.
- El archivo resultante se descarga con el nombre VTT_{POL}_{POD}_{SHIPPER}.png.

14. EXPORTACION A EXCEL

14.1. Hoja Timeline

La funcion build_excel_workbook crea una hoja "Timeline" con:

- POL y POD.
- Fila de informacion con ID-Cartography, Carrier, Shipper, ILN/FF, PLANT, Commodity y E/D.
- La tabla timeline con semanas, fechas, valores y colores.
- El bloque VTT SUMMARY como mini Gantt.
- Customer Safety STOCK.

14.2. Hoja visual "UI Snapshot"

El flujo real del boton Excel no usa la opcion server-side de snapshot. En cambio:

- Primero genera el workbook base en Python con include_snapshot_sheet=False.
- Luego, en el navegador, html2canvas captura timeline_capture.
- ExcelJS abre el workbook base.
- Agrega una hoja nueva llamada UI Snapshot.
- Inserta ahi la imagen PNG de la captura.
- Finalmente descarga el Excel completo.

Resultado final del Excel descargado:

- Hoja Timeline con estructura numerica/operativa.
- Hoja UI Snapshot con la captura visual de la pantalla.

15. TOLERANCIA A DATOS FALTANTES Y VARIANTES DE NOMBRE

El script contempla varias diferencias de nombres o valores nulos:

- Commodity o Comodity.
- 12 Customs Clearance o 12 Customs clearence.
- 9 ETD> ETA o 9 ETD>ETA.
- Round. o Round.

Si una celda viene vacia o con texto mixto, `_coerce_to_int` intenta rescatar el numero con regex. Si no puede, devuelve 0.

16. RESUMEN CORTO DEL COMPORTAMIENTO

En terminos simples, VTT2.py hace esto:

- Lee una fila del Excel segun POL y POD.
- Muestra sus datos generales.
- Convierte varias columnas del Excel en una cadena de pasos logística.
- Usa Day, Day+ y Final Day para pintar un calendario relativo.
- Calcula KPIs derivados sobre esa misma secuencia.
- Permite descargar una version Excel y una imagen de la vista.

17. PUNTOS IMPORTANTES A TENER EN CUENTA SI SE MODIFICA EL EXCEL O EL CODIGO

- Cambiar el orden de columnas puede romper Shipper e ILN/FF porque se leen por indice, no por nombre.
- Cambiar nombres de columnas puede romper pasos completos si el codigo no contempla ese alias.
- La logica KPI debe seguir centralizada en `_build_kpi_rows` para que UI y Excel no diverjan.
- El step 16 Manufacturing existe como soporte parcial, pero no se renderiza mientras no se agregue a `time_labels`.